

Trabajo Práctico 1 — Primer juego de texto

Diplomatura de Videojuegos · Semana 1 (clases 1 y 2) Objetivo: practicar **variables**, **arreglos**, **if**, **for**, **funciones** e **input** con **una escena por tema**, igual que en clase, y al final juntarlos en un combate por consola: *La Cripta del Golem*.

Al finalizar este TP

Seis escenas chicas, una por concepto, cada una con su script y su salida en el panel **Output**. Y una séptima que las junta:

```
=== LA CRIPTA DEL GOLEM ===
Aria se enfrenta a un Golem de piedra!
Tu arsenal:
  [0] Espada (daño 25)
  [1] Arco (daño 15)
  [2] Hacha (daño 40)
Controles: ← →  cambiar arma  |  ENTER  atacar  |  ↑  poción
---
Aria: 100 HP  |  Golem de piedra: 120 HP  |  Pociones: 2
---
```

 **Tiempo estimado: 60–90 min.** Leer cada paso completo antes de hacerlo, y **escribir el código a mano**: se aprende tecleando y rompiendo.

Cómo va a quedar el proyecto

Todo vive en **un solo proyecto**, **tp1**: siete escenas, cada una con su script.

```
res://
├─ 01_variables.tscn      01_variables.gd      ← Parte 1
├─ 02_arreglos.tscn       02_arreglos.gd       ← Parte 2
├─ 03_condicionales.tscn  03_condicionales.gd  ← Parte 3
├─ 04_for.tscn            04_for.gd            ← Parte 4
├─ 05_funciones.tscn      05_funciones.gd      ← Parte 5
├─ 06_input.tscn          06_input.gd          ← Parte 6
└─ 07_cripta.tscn         07_cripta.gd         ← Parte 7 (el juego)
```

🔗 Quien haya hecho el proyecto `clase-02` en la teórico-práctica ya conoce el ritual de crear una escena con un script. Acá se repite en un proyecto nuevo, con los nombres de arriba.

🗨 Antes de arrancar: ¿por qué “en la consola”?

En la clase se vio que `print()` es la mejor herramienta para depurar. Acá se usa como *pantalla*: en vez de mover un personaje, se **imprime** lo que pasa. Así la atención queda en la **lógica** (variables, decisiones, repeticiones), sin pelear con gráficos.

Lo único “gráfico” es el **input**: las teclas se leen con el sistema `Input` de Godot, pero la imagen del juego es el texto que aparece en el panel **Output**.

🔄 Parte 0 — El ritual: una escena, un nodo, un script

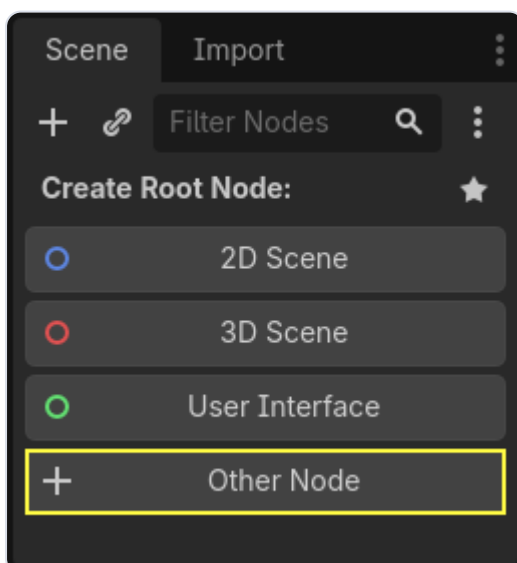
Cada parte de acá en adelante arranca **igual**, con estos pasos. La primera vez se hacen completos; después el TP solo indica “*el ritual, con tal nombre*”.

Crear el proyecto (solo la primera vez)

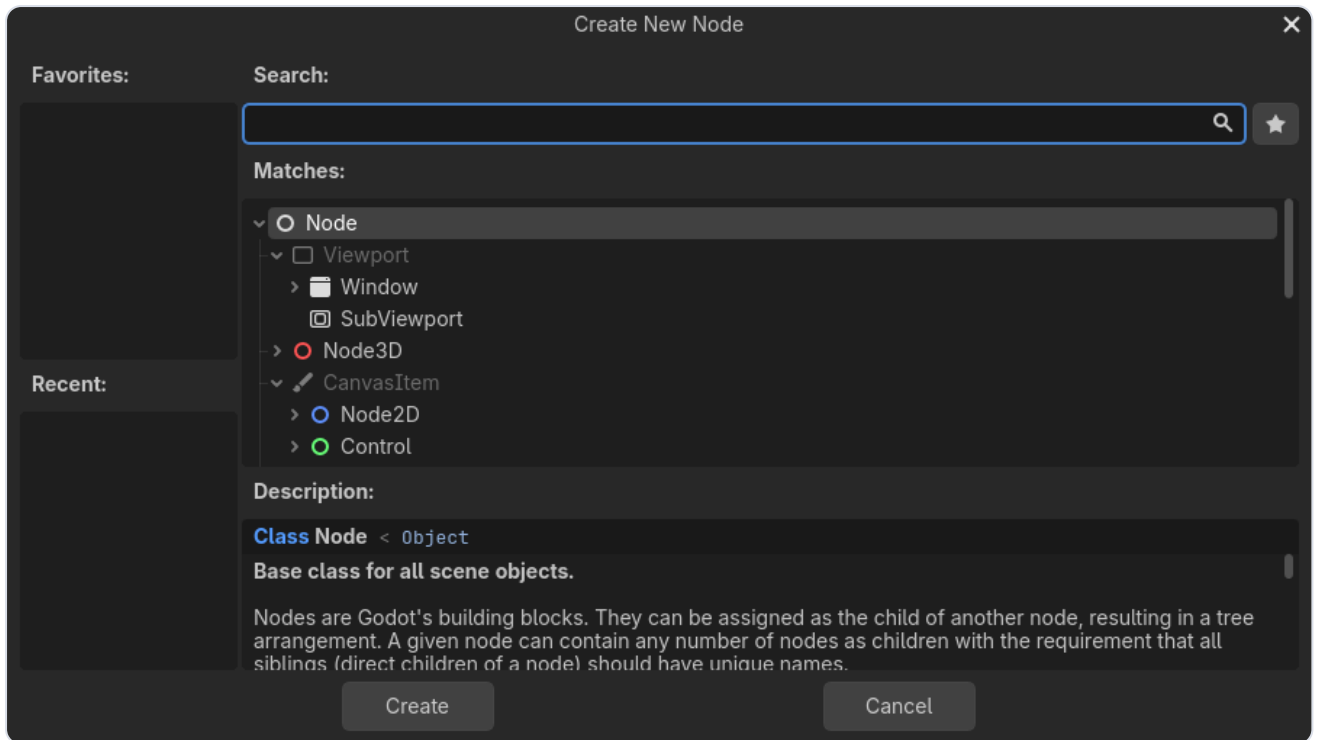
1. Abrir Godot. En el **Project Manager**, clic en **New Project** (Nuevo proyecto).
2. Nombre: `tp1`. Elegir una carpeta **vacía**. Dejar el renderizador que viene por defecto → **Create & Edit**.

El ritual (en cada parte)

3. Con el editor abierto, en el panel **Escena** (arriba a la izquierda), clic en **Otro Nodo** (*Other Node*). Para las escenas siguientes, primero crear una escena nueva con el menú **Scene** → **New Scene** (**Ctrl + N**).

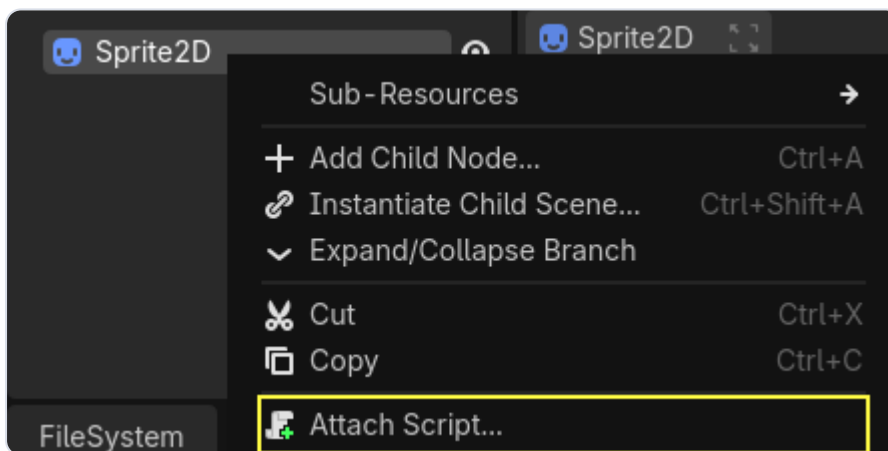


4. Buscar `Node` (el nodo más básico, sin nada visual), seleccionarlo y **Crear**.

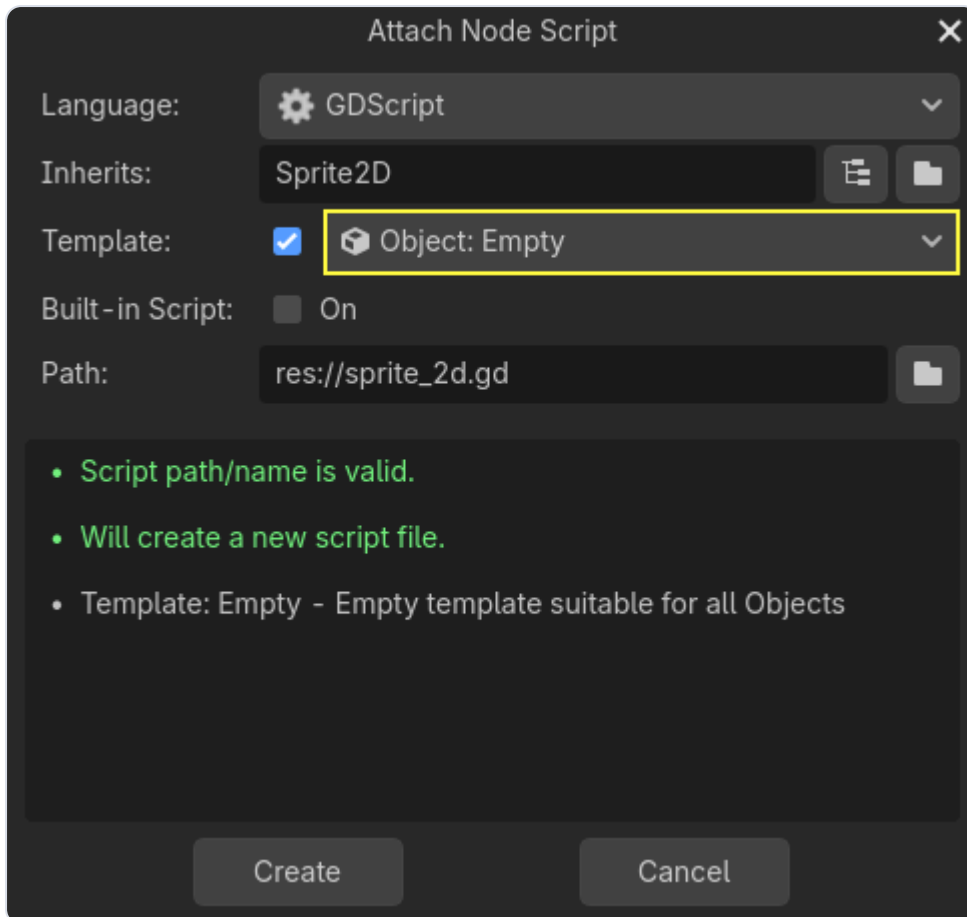


5. Renombrarlo (doble clic sobre el nombre). En esta primera escena: `Variables`.

6. **Adjuntar un script:** clic derecho sobre el nodo → **Attach Script** (*Adjuntar Script*), o el botón con el ícono 📄 arriba del panel Escena.



7. En el diálogo, dejar **Language: GDScript** y cambiar el **Path** a `res://01_variables.gd`. Clic en **Create**.



8. Se abre el **editor de Script** con una plantilla. **Borrar todo** y dejar solo esto:

```
extends Node

func _ready():
    print("¡La escena arrancó!")
```

9. Guardar con **Ctrl + S**. Godot pide el nombre de la escena: `01_variables.tscn`.

10. **Ejecutar la escena actual** con **F6** (no F5: F5 corre la escena principal del proyecto, y si no hay una definida, Godot pide elegirla). Mirar el panel inferior **Output** (*Salida*): tiene que aparecer el mensaje.

```
Godot Engine v4.6.rc1.official
D3D12 12_0 - Forward+ - Using
1060 6GB

Hello, world!
--- Debugging process stopped
```

 `extends Node` dice qué tipo de nodo controla el script. `func _ready():` se ejecuta **una sola vez** al aparecer el nodo. Todo lo que se imprima ahí aparece en **Output**. Y si al correr se abre una ventana gris y vacía, **está bien**: el juego "es" lo que sale en la consola.

✓ **Punto de control 0:** "¡La escena arrancó!" aparece en el panel Output al apretar F6, y en el FileSystem están `01_variables.tscn` y `01_variables.gd`.

No aparece nada / da error

▼ Ver soluciones

- La **sangría importa**: lo de adentro de `func` va corrido con **un Tab**. No mezclar tabs y espacios.
- ¿Se guardó con **Ctrl + S** antes de apretar F6?
- ¿Se apretó **F6** (escena actual) y no F5?
- ¿El panel de abajo está en la pestaña **Output**, no en **Debugger**?

Parte 1 — Variables: la ficha de la heroína

Escena: `01_variables` (la creada en la Parte 0). **Concepto:** asignación de variables (`var`) y tipos (`int` , `float` , `String` , `bool`), imprimir con `str()` .

Reemplazar el `func _ready()` de `01_variables.gd` por esto y apretar **F6**:

```
extends Node

func _ready():
    var nombre = "Aria"          # String  (texto)
    var vida = 100               # int     (entero)
    var precision = 85.5         # float   (decimal)
    var es_heroína = true        # bool    (verdadero/falso)

    print("Nombre: " + nombre)
    print("Vida: " + str(vida))
    print("Precisión: " + str(precision) + "%")
    print("¿Es heroína?: " + str(es_heroína))
```

Salida esperada en Output:

```
Nombre: Aria
Vida: 100
Precisión: 85.5%
¿Es heroína?: true
```

 Una variable es una **caja con etiqueta**: `vida` guarda un `100` . Para **pegar** un número con texto hay que convertirlo con `str()` : `"Vida: " + str(vida)` . Sin el `str()` , Godot marca un error en rojo: no sabe sumar texto con número.

 **Para probar:** cambiar los valores. Poner un nombre propio y una vida distinta. Quitarle el `str()` a `vida` , correr y leer el error: es uno de los que más aparecen.

✅ **Punto de control 1:** se imprime una ficha con los 4 tipos de datos.

🎒 Parte 2 — Arreglos: el arsenal

Escena: el ritual, con nodo `Arreglos`, script `02_arreglos.gd` y escena `02_arreglos.tscn`. **Concepto:** arreglos (arrays): crear, acceder por **índice**, tamaño con `.size()`, agregar con `.append()`.

Un **arreglo** es una variable que guarda **varias cosas en orden**. Cada elemento tiene un **número de posición** (índice) que **empieza en 0**.

```
extends Node

func _ready():
    var armas = ["Espada", "Arco", "Hacha"]

    print("Primer arma: " + armas[0])      # índice 0 → Espada
    print("Segunda arma: " + armas[1])    # índice 1 → Arco
    print("Cuántas armas tengo: " + str(armas.size()))

    armas.append("Daga")                  # agrega al final
    print("Nueva arma agregada: " + armas[3])
    print("Ahora tengo: " + str(armas.size()) + " armas")
```

Output:

```
Primer arma: Espada
Segunda arma: Arco
Cuántas armas tengo: 3
Nueva arma agregada: Daga
Ahora tengo: 4 armas
```

💡 **El índice empieza en 0.** En un arreglo de 3 elementos, las posiciones son `0`, `1` y `2`. Pedir `armas[3]` **antes** de agregar la Daga daría error ("índice fuera de rango"): esa posición todavía no existe.

💡 **Truco de diseño: arreglos en paralelo.** El **daño** de cada arma se guarda en **otro** arreglo, en el mismo orden. `armas[2]` es `"Hacha"` y `danos[2]` es su daño. Así se usan en el juego final. Agregarlo al final del `_ready()`:

```
var danos = [25, 15, 40, 10]
print(armas[2] + " hace " + str(danos[2]) + " de daño")  # Hacha hace 40 de daño
```

✓ **Punto de control 2:** se accede a elementos por índice y se usan `.size()` y `.append()`.

Parte 3 — Condicionales: decidir el resultado

Escena: el ritual, con nodo `Condicionales`, script `03_condicionales.gd` y escena `03_condicionales.tscn`. **Concepto:** `if` / `elif` / `else` y operadores de comparación (`==`, `!=`, `<`, `>`, `<=`, `>=`).

```
extends Node

func _ready():
    var vida_enemigo = 40
    var dano = 25

    vida_enemigo = vida_enemigo - dano
    print("Vida del enemigo: " + str(vida_enemigo))

    if vida_enemigo <= 0:
        print("💀 El enemigo cayó")
    elif vida_enemigo <= 20:
        print("🔴 El enemigo está por morir")
    else:
        print("🟢 El enemigo sigue fuerte")
```

Output (con estos valores):

```
Vida del enemigo: 15
🔴 El enemigo está por morir
```

💡 Godot revisa las condiciones **de arriba hacia abajo** y ejecuta **la primera que sea verdadera**. Con `vida_enemigo = 15`: no es `<= 0`, **sí** es `<= 20` → imprime la de vida crítica y **saltea el resto**.

🎯 **Para probar:** cambiar `dano` a `50` (¿qué rama sale?) y a `10` (¿y ahora?). Predecir el resultado **antes** de apretar F6.

✓ **Punto de control 3:** según el daño, cae en una rama distinta.

Parte 4 — `for` : repetir sin copiar y pegar

Escena: el ritual, con nodo `For`, script `04_for.gd` y escena `04_for.tscn`. **Concepto:** el loop `for`, con `range()` y recorriendo un arreglo.

```
extends Node

func _ready():
    # 1) Repetir N veces con range()
    print("--- Aparecen enemigos ---")
    for i in range(3):
        print("Golem #" + str(i + 1) + " entró a la cripta")

    # 2) Recorrer un arreglo elemento por elemento
    print("--- Tu arsenal ---")
    var armas = ["Espada", "Arco", "Hacha"]
    for arma in armas:
        print("Tenés: " + arma)

    # 3) Recorrer con el índice (para arreglos en paralelo)
    var danos = [25, 15, 40]
    for i in range(armas.size()):
        print "[" + str(i) + "] " + armas[i] + " → daño " + str(danos[i]))
```

Output:

```
--- Aparecen enemigos ---
Golem #1 entró a la cripta
Golem #2 entró a la cripta
Golem #3 entró a la cripta
--- Tu arsenal ---
Tenés: Espada
Tenés: Arco
Tenés: Hacha
[0] Espada → daño 25
[1] Arco → daño 15
[2] Hacha → daño 40
```

💡 Hay dos formas de recorrer: `for arma in armas` da directo cada valor (`"Espada"` , `"Arco"` ...). `for i in range(armas.size())` da el **número de posición** (`0` , `1` , `2`), útil cuando hace falta el índice para leer **dos arreglos en paralelo** al mismo tiempo.

⚠ **Importante (de la clase):** en Godot los `for` **no** se usan para la lógica en tiempo real (para eso está `_process()`). Sirven para **inicializar**, generar contenido y recorrer datos, justo lo que se hizo acá.

🔗 **Para probar:** `range(10)` . Agregar `"Arco largo"` a `armas` : ¿hace falta tocar el `for` ? ¿Y el tercer `for` sigue andando? (Pista: `danos` quedó más corto.)

✅ **Punto de control 4:** se imprimen listas con `for` , con y sin índice.

🧰 Parte 5 — Funciones: escribir una vez, usar mil

Escena: el ritual, con nodo `Funciones` , script `05_funciones.gd` y escena `05_funciones.tscn` . **Concepto:** crear funciones, pasarles **parámetros** y devolver un valor con `return` .

```
extends Node

var vida = 100    # variable "global" del script: la ven todas las funciones

func _ready():
    mostrar_estado()
    recibir_dano(30)    # se le pasa un parámetro
    recibir_dano(50)
    if esta_vivo():      # se usa lo que devuelve
        print("Aria sigue en pie")
    else:
        print("Aria cayó")

# Recibe un parámetro y modifica la vida
func recibir_dano(cantidad):
    vida = vida - cantidad
    print("Recibí " + str(cantidad) + " de daño. Vida: " + str(vida))

# Devuelve un valor (true o false)
func esta_vivo():
    return vida > 0

func mostrar_estado():
    print("=== Vida actual: " + str(vida) + " ===")
```

Output:

```
=== Vida actual: 100 ===
Recibí 30 de daño. Vida: 70
Recibí 50 de daño. Vida: 20
Aria sigue en pie
```

Anatomía:

- `func recibir_dano(cantidad):` → **definición**. `cantidad` es el **parámetro** (el dato que entra).
- `recibir_dano(30)` → **llamado**. Los `()` son **obligatorios**.
- `return vida > 0` → **devuelve** un resultado (acá `true` o `false`) que se puede usar en un `if`.

Notar que `vida` está declarada **afuera** de las funciones (arriba de todo). Por eso **todas** la pueden leer y modificar. Es el “estado” del juego.

 **Para probar:** un tercer `recibir_dano(30)` : ¿qué imprime el `if` ? Escribir `curar(cantidad)` , que suma vida, y llamarla.

 **Punto de control 5:** hay funciones definidas con parámetro y con `return` , y se las llama.

Parte 6 — Input: que reaccione a las teclas

Escena: el ritual, con nodo `Input` , script `06_input.gd` y escena `06_input.tscn` . **Concepto:** detectar input con `Input.is_action_just_pressed(...)` dentro de `_process()` .

Hasta ahora todo corría solo en `_ready()` . Para **reaccionar al jugador** hace falta `_process()` , que corre en **cada frame** (~60 por segundo).

```
extends Node

func _ready():
    print("Presioná: ENTER (atacar), ← →, o ↑")

func _process(delta):
    if Input.is_action_just_pressed("ui_accept"):    # Enter / Espacio
        print("¡Ataque!")
    if Input.is_action_just_pressed("ui_right"):     # flecha →
        print("Elegiste el arma de la derecha")
    if Input.is_action_just_pressed("ui_left"):      # flecha ←
        print("Elegiste el arma de la izquierda")
    if Input.is_action_just_pressed("ui_up"):        # flecha ↑
        print("Tomaste una poción")
```

Apretar **F6**. **Importante:** hacer **clic sobre la ventana del juego** para que reciba las teclas; los mensajes aparecen en el panel **Output** del editor.

 `is_action_just_pressed` se dispara **una sola vez** por pulsación (ideal para menús y ataques). `is_action_pressed` es `true` **mientras** la tecla está apretada (ideal para movimiento continuo). Las acciones `ui_accept`, `ui_left`, `ui_right`, `ui_up`, `ui_down`, `ui_cancel` **ya vienen** definidas; se pueden crear otras en `Project → Project Settings → Input Map`.

 **Para probar:** cambiar el `just_pressed` del ataque por `pressed` y mantener apretado ENTER. Eso es `_process()` : 60 veces por segundo.

✓ **Punto de control 6:** cada tecla imprime su mensaje en Output.

 **Las teclas no hacen nada**

▼ Ver soluciones

- ¿Se hizo **clic en la ventana del juego**? Si el foco está en el editor, no llegan las teclas.
- El código de input va en `_process(delta)`, no en `_ready()`.
- Revisar que `ui_accept` esté bien escrito (con guion bajo) y entre comillas.

Parte 7 — Todo junto: *La Cripta del Golem*

Escena: el ritual, con nodo `Cripta`, script `07_cripta.gd` y escena `07_cripta.tscn`.

Ahora se combinan **las seis piezas** en un solo juego. Escribirlo leyéndolo: cada bloque indica **qué parte del TP usa** (están marcados con comentarios).

```

extends Node

# ---- VARIABLES: estado del juego (Parte 1) ----
var nombre = "Aria"
var vida = 100
var pociones = 2
var terminado = false

# ---- ARREGLOS en paralelo (Parte 2) ----
var armas = ["Espada", "Arco", "Hacha"]
var danos = [25, 15, 40]
var arma_actual = 0          # índice del arma elegida

# ---- Enemigo ----
var enemigo = "Golem de piedra"
var vida_enemigo = 120

func _ready():
    print("=== LA CRIPTA DEL GOLEM ===")
    print(nombre + " se enfrenta a un " + enemigo + "!")
    mostrar_inventario()      # usa un FOR (Parte 4)
    print("Controles: ← → cambiar arma | ENTER atacar | ↑ poción")
    mostrar_estado()

func _process(delta):
    # INPUT (Parte 6). Si el juego terminó, no se hace nada.
    if terminado:
        return

    if Input.is_action_just_pressed("ui_right"):
        arma_actual = arma_actual + 1
        if arma_actual >= armas.size(): # si se pasa del final, vuelve al principio
            arma_actual = 0
        print("Arma seleccionada: " + armas[arma_actual])

    if Input.is_action_just_pressed("ui_left"):
        arma_actual = arma_actual - 1
        if arma_actual < 0:
            arma_actual = armas.size() - 1
        print("Arma seleccionada: " + armas[arma_actual])

    if Input.is_action_just_pressed("ui_accept"):
        atacar()

    if Input.is_action_just_pressed("ui_up"):
        curar()

# ---- FUNCIONES (Parte 5) ----

func mostrar_inventario():

```

```

print("Tu arsenal:")
for i in range(armas.size()):          # FOR con índice
    print("  [" + str(i) + "] " + armas[i] + " (daño " + str(danos[i]) + ")")

func mostrar_estado():
    print("---")
    print(nombre + ": " + str(vida) + " HP   |   " + enemigo + ": " + str(vida_enemigo) + " HP
    | Pociones: " + str(pociones))
    print("---")

func atacar():
    var dano = danos[arma_actual]      # lee el arreglo por índice
    print(nombre + " ataca con " + armas[arma_actual] + " (" + str(dano) + " de daño)")
    vida_enemigo = vida_enemigo - dano

    if vida_enemigo <= 0:               # CONDICIONAL (Parte 3)
        print("🏆 ¡Derrotaste al " + enemigo + "! GANASTE.")
        terminado = true
        return

    enemigo_contraataca()
    mostrar_estado()

func enemigo_contraataca():
    var golpe = 20
    print("El " + enemigo + " contraataca: -" + str(golpe) + " HP")
    vida = vida - golpe
    if vida <= 0:
        print("💀 " + nombre + " cayó. GAME OVER.")
        terminado = true

func curar():
    if pociones <= 0:
        print("No te quedan pociones.")
        return
    pociones = pociones - 1
    vida = vida + 30
    print(nombre + " bebe una poción: +30 HP")
    mostrar_estado()

```

Cómo se juega: apretar **F6**, hacer **click en la ventana del juego**, y usar ← → para elegir arma, **ENTER** para atacar y ↑ para curarse. El combate se desarrolla en el panel **Output**.

🗨 **return solo, sin valor**, corta la función ahí mismo. En `_process()` se usa para que, cuando el juego terminó, no siga leyendo teclas. En `curar()`, para no restar pociones que ya no quedan.

✅ **Punto de control 7 (final):** se le puede ganar al Golem eligiendo el Hacha y atacando, o perder por descuido. Todo se ve en Output.

🛑 **Errores comunes al armar el juego final**

▼ Ver soluciones

- “Invalid index / índice fuera de rango”: `armas` y `danos` tienen que tener **la misma cantidad** de elementos.
- **Sangría mezclada**: usar Tab en todo el archivo, sin combinarlo con espacios.
- **Se repite el ataque sin parar**: usar `is_action_just_pressed` (una vez por pulsación), no `is_action_pressed`.
- **No pasa nada al apretar teclas**: clic en la ventana del juego para darle foco.
- “Identifier not found: atacar”: la función está mal escrita o le falta el `func` adelante. Los nombres tienen que coincidir letra por letra.

Entrega

Entregar **una** de estas opciones (según indique el/la docente):

1. La **carpeta del proyecto** `tp1` comprimida en `.zip` (sin la carpeta `.godot/`), o
2. **Dos capturas**: el panel **FileSystem** con las siete escenas y sus scripts, y el panel **Output** mostrando una partida de la Cripta (un ataque, una curación y el final).

Nombre del archivo: `tp1-ApellidoNombre.zip`

✓ Checklist de autoevaluación

- ☐ Hay **siete escenas** (`01_variables` a `07_cripta`), cada una con un `Node` y su script adjunto, y todas imprimen en **Output**.
- ☐ Se usan **variables** de al menos 3 tipos distintos (`int` , `String` , `bool` ...).
- ☐ Se usa un **arreglo** y se accede a un elemento por **índice**.
- ☐ Se usa `if` / `elif` / `else` para decidir un resultado.
- ☐ Se usa un `for` para recorrer el arsenal.
- ☐ Hay al menos **2 funciones** definidas (una con **parámetro**, una con `return`).
- ☐ El juego reacciona a **teclas** con `Input.is_action_just_pressed` .
- ☐ Se puede **ganar** y **perder** el combate.

Extra (opcional, para quien quiera más)

- **Daño variable (azar)**: hacer que cada golpe varíe un poco. `randi_range(min, max)` devuelve un entero al azar:

```
var dano = danos[arma_actual] + randi_range(-5, 5)
```

- **Segundo enemigo:** guardar los enemigos en arreglos en paralelo (`enemigos = [...]` , `vidas_enemigo = [...]`) y pasar al siguiente con un índice cuando cae uno.
- **Función `esta_vivo()`** que devuelva `vida > 0` , para usarla en los `if` en vez de comparar a mano.
- **Estadísticas al final:** contar cuántos turnos duró el combate con una variable `turnos` que suba en cada ataque, e imprimirla al ganar o perder.
- **Otro tema, misma estructura:** este mismo esqueleto (variables + arreglos + `if` + `for` + funciones + input) sirve para una **aventura de texto** (se eligen caminos con las flechas), un **selector de pociones** o un **gestor de recursos** tipo estrategia. Cambia el tema, no la lógica.

Recursos

- Crear el primer script (capturas de este TP): [Creating your first script](#)
- Fundamentos de GDScript: [GDScript basics](#)
- Arreglos (`Array`): [referencia de la clase Array](#)
- Assets gratis (CC0): [kenney.nl/assets](#)

Las capturas de este documento provienen de la **documentación oficial de Godot Engine** (Juan Linietsky, Ariel Manzur y la comunidad), bajo licencia **CC BY 4.0**.